

TALLER PRÁCTICO: Consultas y optimización en PostgreSQL con Chinook

Consultas básicas, JOIN, EXPLAIN ANALYZE e índices

Integrantes SERGIO APARICIO, JHON REYES

Información General

Campo	Detalle
Duración	1 hora y 30 minutos
Herramientas	PostgreSQL y pgAdmin Query Tool
Base de datos	Chinook
Modalidad	Individual o en parejas

Datos del estudiante

- **Nombre:** [Nombre del estudiante]
- **Grupo:** [Grupo / Curso]
- **Fecha:** 15 de septiembre de 2026

Propósito del taller

En este taller analizarás información de la tienda musical Chinook mediante consultas SQL. Construirás filtros, agregaciones y relaciones entre tablas; después compararás planes de ejecución y aplicarás índices para determinar cuándo una optimización resulta conveniente.

Resultado de aprendizaje

Al finalizar, podrás formular consultas de negocio con PostgreSQL, relacionar correctamente las tablas de Chinook y justificar una decisión de optimización con evidencia obtenida mediante `EXPLAIN ANALYZE`.

Distribución del tiempo

Etapas	Actividad	Tiempo
1	Reconocimiento de la base de datos	10 min
2	Consultas básicas	20 min
3	Consultas con JOIN	25 min
4	Análisis y optimización	25 min

Etapas	Actividad	Tiempo
5	Reto final y conclusiones	10 min
Total		90 min

Indicaciones técnicas

- Trabajar en la base de datos donde se cargó el script de Chinook (`chinook_pgadmin_query_tool.sql`).
- Ejecutar cada consulta desde Query Tool y conservar las consultas solicitadas en un archivo SQL.
- Usar comillas dobles en nombres como `"Track"` o `"TrackId"`, ya que fueron definidos respetando mayúsculas y minúsculas (CamelCase).
- Antes de crear un índice, registrar el plan de ejecución inicial para poder compararlo.
- No evaluar una optimización únicamente por el tiempo de reloj: revisar el tipo de recorrido, el costo y las filas procesadas.

Tablas principales del modelo Chinook

Tabla	Contenido
Artist	Artistas e intérpretes musicales
Album	Álbumes y producciones discográficas
Track	Canciones y pistas de audio
Genre	Géneros musicales
MediaType	Tipos y formatos de archivo multimedia
Customer	Clientes registrados de la tienda
Employee	Empleados de la empresa y estructura de reporte
Invoice	Facturas y cabeceras de compras
InvoiceLine	Detalles y líneas de productos comprados en cada factura
Playlist	Listas de reproducción de música
PlaylistTrack	Canciones asociadas a cada lista de reproducción

Etapas 1: Reconocimiento de la base de datos

Tiempo sugerido: 10 minutos

Actividad 1: Explorar las tablas

Consultas para inspeccionar los primeros registros de las tablas solicitadas:

```
-- Exploración de Artist
SELECT *
FROM public."Artist"
LIMIT 10;

-- Exploración de Album
SELECT *
FROM public."Album"
LIMIT 10;

-- Exploración de Track
SELECT *
FROM public."Track"
LIMIT 10;

-- Exploración de Customer
SELECT *
FROM public."Customer"
LIMIT 10;

-- Exploración de Invoice
SELECT *
FROM public."Invoice"
LIMIT 10;

-- Exploración de InvoiceLine
SELECT *
FROM public."InvoiceLine"
LIMIT 10;
```

Actividad 2: Contar registros

Consultas para contabilizar el total de filas presentes en cada tabla del esquema Chinook:

```
-- Conteo de registros por tabla
SELECT 'Artist' AS tabla, COUNT(*) AS cantidad_registros FROM public."Artist"
UNION ALL
SELECT 'Album', COUNT(*) FROM public."Album"
UNION ALL
SELECT 'Track', COUNT(*) FROM public."Track"
UNION ALL
SELECT 'Customer', COUNT(*) FROM public."Customer"
UNION ALL
SELECT 'Invoice', COUNT(*) FROM public."Invoice"
UNION ALL
SELECT 'InvoiceLine', COUNT(*) FROM public."InvoiceLine"
UNION ALL
SELECT 'Genre', COUNT(*) FROM public."Genre"
UNION ALL
```

```
SELECT 'MediaType', COUNT(*) FROM public."MediaType"  
UNION ALL  
SELECT 'Employee', COUNT(*) FROM public."Employee"  
UNION ALL  
SELECT 'Playlist', COUNT(*) FROM public."Playlist"  
UNION ALL  
SELECT 'PlaylistTrack', COUNT(*) FROM public."PlaylistTrack";
```

Cantidad exacta de registros en Chinook:

- **Artist:** 275 registros
- **Album:** 347 registros
- **Track:** 3,503 registros
- **Customer:** 59 registros
- **Invoice:** 412 registros
- **InvoiceLine:** 2,240 registros
- (Adicionales: *PlaylistTrack: 8,715; Playlist: 18; Genre: 25; MediaType: 5; Employee: 8*)

Preguntas de reconocimiento

1. ¿Cuál es la tabla con más registros?

Respuesta:

Dentro de las tablas solicitadas en la actividad, la tabla con mayor volumen de filas es **Track** con **3,503 registros**, seguida de **InvoiceLine** con **2,240 registros**.
(Si consideramos la totalidad de la base de datos, la tabla intermedia **PlaylistTrack** contiene la mayor cantidad global con **8,715 registros**).

2. ¿Qué columna relaciona **Album** con **Artist**?

Respuesta:

La columna es **"ArtistId"**. En la tabla **public."Album"** actúa como clave foránea (*Foreign Key*) referenciando a la clave primaria **public."Artist"("ArtistId")**.

3. ¿Qué tablas permiten conocer las canciones compradas en una factura?

Respuesta:

Se requieren las tablas **InvoiceLine** y **Track** (relacionadas mediante **"TrackId"**).
Para vincular la información directamente con los metadatos de la factura (como la fecha y el cliente), se incluye también **Invoice** (relacionada con **InvoiceLine** mediante **"InvoiceId"**).
La relación completa es: **Invoice** $\rightarrow$ **InvoiceLine** $\rightarrow$ **Track**.

4. ¿Cuál es la diferencia entre **Invoice** e **InvoiceLine**?

Respuesta:

- **Invoice (Cabecera de factura):** Modela la transacción general a nivel macro. Almacena un registro único por compra donde se registra el código de factura (`InvoiceId`), el cliente comprador (`CustomerId`), la fecha (`InvoiceDate`), la dirección de facturación (`BillingAddress`, `BillingCountry`, etc.) y el importe total facturado (`Total`).
- **InvoiceLine (Detalle o renglón de factura):** Modela los ítems individuales adquiridos dentro de una factura. Posee una relación de 1 a N con `Invoice`, almacenando qué canción específica se adquirió (`TrackId`), a qué precio unitario en el momento de la venta (`UnitPrice`) y en qué cantidad (`Quantity`).

Etapla 2: Consultas básicas

Tiempo sugerido: 20 minutos

Ejercicio 1: Filtros y ordenamiento

Obtén las canciones cuyo precio sea mayor o igual a 1.00. Muestra el identificador, el nombre y el precio, ordenando desde la canción más costosa.

```
SELECT
    "TrackId",
    "Name",
    "UnitPrice"
FROM public."Track"
WHERE "UnitPrice" >= 1.00
ORDER BY "UnitPrice" DESC, "Name" ASC;
```

Ejercicio 2: Búsqueda de clientes

Consulta los clientes de Brasil, Canadá o Estados Unidos. Muestra nombre completo, país, ciudad y correo electrónico. Utiliza `IN` para el filtro.

```
SELECT
    "FirstName" || ' ' || "LastName" AS "NombreCompleto",
    "Country",
    "City",
    "Email"
FROM public."Customer"
WHERE "Country" IN ('Brazil', 'Canada', 'USA')
ORDER BY "Country" ASC, "NombreCompleto" ASC;
```

Ejercicio 3: Búsqueda de canciones

Encuentra las canciones cuyo nombre contenga la palabra *Love*, sin importar mayúsculas o minúsculas.

```
SELECT
    "TrackId",
    "Name",
    "Composer",
    "UnitPrice"
FROM public."Track"
WHERE "Name" ILIKE '%love%'
ORDER BY "Name" ASC;
```

Ejercicio 4: Funciones de agregación

Obtén en una sola consulta:

- Cantidad total de canciones.
- Precio promedio, mínimo y máximo.
- Duración promedio en milisegundos.

Debes emplear `COUNT`, `AVG`, `MIN` y `MAX`.

```
SELECT
    COUNT(*) AS total_canciones,
    ROUND(AVG("UnitPrice")::numeric, 2) AS precio_promedio,
    MIN("UnitPrice") AS precio_minimo,
    MAX("UnitPrice") AS precio_maximo,
    ROUND(AVG("Milliseconds")::numeric, 2) AS duracion_promedio_ms
FROM public."Track";
```

Ejercicio 5: Agrupación

Obtén la cantidad de clientes por país y ordena el resultado desde el país con más clientes.

```
SELECT
    "Country" AS pais,
    COUNT(*) AS cantidad_clientes
FROM public."Customer"
GROUP BY "Country"
ORDER BY cantidad_clientes DESC, pais ASC;
```

Ejercicio 6: Condición sobre agrupaciones

Modifica el ejercicio anterior para mostrar únicamente los países que tengan al menos dos clientes (`HAVING COUNT(*) >= 2`).

```
SELECT
    "Country" AS pais,
    COUNT(*) AS cantidad_clientes
FROM public."Customer"
GROUP BY "Country"
HAVING COUNT(*) >= 2
ORDER BY cantidad_clientes DESC, pais ASC;
```

Etapa 3: Consultas con JOIN

Tiempo sugerido: 25 minutos

Ejercicio 7: Álbumes y artistas

Obtén cada álbum junto con el nombre de su artista.

```
SELECT
    al."AlbumId",
    al."Title" AS album,
    ar."Name" AS artista
FROM public."Album" al
INNER JOIN public."Artist" ar ON al."ArtistId" = ar."ArtistId"
ORDER BY ar."Name" ASC, al."Title" ASC;
```

Ejercicio 8: Canciones, álbumes y artistas

Relaciona `Track`, `Album` y `Artist`. Muestra canción, álbum, artista, precio y duración en minutos.

```
SELECT
    t."Name" AS cancion,
    al."Title" AS album,
    ar."Name" AS artista,
    t."UnitPrice" AS precio,
    ROUND((t."Milliseconds" / 60000.0)::numeric, 2) AS duracion_minutos
FROM public."Track" t
INNER JOIN public."Album" al ON t."AlbumId" = al."AlbumId"
INNER JOIN public."Artist" ar ON al."ArtistId" = ar."ArtistId"
ORDER BY ar."Name" ASC, al."Title" ASC, t."Name" ASC;
```

Ejercicio 9: Clientes y facturas

Obtén todas las facturas junto con el cliente. Muestra número de factura, nombre completo, país, fecha y total. Ordena desde la factura más reciente.

```
SELECT
    i."InvoiceId" AS numero_factura,
    c."FirstName" || ' ' || c."LastName" AS nombre_completo,
    c."Country" AS pais,
    i."InvoiceDate" AS fecha,
    i."Total" AS total
FROM public."Invoice" i
INNER JOIN public."Customer" c ON i."CustomerId" = c."CustomerId"
ORDER BY i."InvoiceDate" DESC;
```

Ejercicio 10: Detalle completo de ventas

Relaciona `Customer`, `Invoice`, `InvoiceLine` y `Track` para mostrar cada canción vendida.

```
SELECT
    c."FirstName" || ' ' || c."LastName" AS cliente,
    i."InvoiceId" AS numero_factura,
    t."Name" AS cancion,
    il."UnitPrice" AS precio_unitario,
    il."Quantity" AS cantidad,
    (il."UnitPrice" * il."Quantity") AS subtotal
FROM public."InvoiceLine" il
INNER JOIN public."Invoice" i ON il."InvoiceId" = i."InvoiceId"
INNER JOIN public."Customer" c ON i."CustomerId" = c."CustomerId"
INNER JOIN public."Track" t ON il."TrackId" = t."TrackId"
ORDER BY i."InvoiceId" ASC, il."InvoiceLineId" ASC;
```

Ejercicio 11: Ventas por país

Calcula el dinero facturado por país. Muestra país de facturación, cantidad de facturas y total vendido. Ordena desde el país con mayores ventas.

```
SELECT
    i."BillingCountry" AS pais_facturacion,
    COUNT(i."InvoiceId") AS cantidad_facturas,
    SUM(i."Total") AS total_vendido
FROM public."Invoice" i
GROUP BY i."BillingCountry"
ORDER BY total_vendido DESC;
```

Ejercicio 12: Cinco artistas con mayores ventas

Relaciona `Artist`, `Album`, `Track` e `InvoiceLine`. Muestra artista, unidades vendidas e ingresos generados. Devuelve únicamente los cinco primeros resultados.

```

SELECT
    ar."Name" AS artista,
    SUM(il."Quantity") AS unidades_vendidas,
    SUM(il."UnitPrice" * il."Quantity") AS ingresos_generados
FROM public."InvoiceLine" il
INNER JOIN public."Track" t ON il."TrackId" = t."TrackId"
INNER JOIN public."Album" al ON t."AlbumId" = al."AlbumId"
INNER JOIN public."Artist" ar ON al."ArtistId" = ar."ArtistId"
GROUP BY ar."ArtistId", ar."Name"
ORDER BY ingresos_generados DESC
LIMIT 5;

```

Etapas 4: Análisis y optimización

Tiempo sugerido: 25 minutos

Lectura del plan de ejecución

- **EXPLAIN**: Muestra la estimación del optimizador (costos estimados, nodos de ejecución proyectados) sin ejecutar físicamente la consulta.
- **EXPLAIN ANALYZE**: Ejecuta la consulta en el motor, descartando la salida de datos y registrando métricas exactas:
 - **Tipo de recorrido**: Seq Scan, Index Scan, Bitmap Index Scan / Bitmap Heap Scan, Index Only Scan.
 - **Costo estimado**: Expresado como (costo_inicio..costo_total) en unidades arbitrarias de lectura de página y CPU.
 - **Filas estimadas vs. Filas reales**: Permite detectar desviaciones estadísticas.
 - **Tiempos**: Planning Time (tiempo consumido en analizar y optimizar el árbol) y Execution Time (tiempo real de procesamiento).

Ejercicio 13: Plan inicial (antes del índice)

Consulta de análisis:

```

EXPLAIN ANALYZE
SELECT
    "TrackId",
    "Name",
    "Composer"
FROM public."Track"
WHERE "Composer" = 'Steve Harris';

```

Plan de ejecución observado (Inicial - sin índice):

```
Seq Scan on "Track" (cost=0.00..83.79 rows=142 width=57) (actual
time=0.024..0.612 rows=142 loops=1)
  Filter: (("Composer")::text = 'Steve Harris')::text)
  Rows Removed by Filter: 3361
Planning Time: 0.115 ms
Execution Time: 0.638 ms
```

Elemento	Resultado antes del índice
Tipo de recorrido	Seq Scan (Recorrido secuencial completo de la tabla)
Filas estimadas	142 filas
Filas reales	142 filas
Costo estimado	cost=0.00..83.79
Tiempo de planificación	~0.115 ms
Tiempo de ejecución	~0.638 ms

Ejercicio 14: Crear y evaluar un índice

Creación del índice B-Tree y actualización de estadísticas:

```
CREATE INDEX idx_track_composer
ON public."Track" ("Composer");

ANALYZE public."Track";
```

Repetición del análisis:

```
EXPLAIN ANALYZE
SELECT
  "TrackId",
  "Name",
  "Composer"
FROM public."Track"
WHERE "Composer" = 'Steve Harris';
```

Plan de ejecución observado (Posterior - con índice):

```
Bitmap Heap Scan on "Track" (cost=5.41..41.83 rows=142 width=57) (actual
time=0.048..0.185 rows=142 loops=1)
  Recheck Cond: (("Composer")::text = 'Steve Harris'::text)
  Heap Blocks: exact=34
  -> Bitmap Index Scan on idx_track_composer (cost=0.00..5.37 rows=142 width=0)
(actual time=0.032..0.032 rows=142 loops=1)
    Index Cond: (("Composer")::text = 'Steve Harris'::text)
Planning Time: 0.218 ms
Execution Time: 0.215 ms
```

Elemento	Resultado después del índice
Tipo de recorrido	Bitmap Heap Scan asistido por Bitmap Index Scan (sobre idx_track_composer)
Filas estimadas	142 filas
Filas reales	142 filas
Costo estimado	cost=5.41..41.83
Tiempo de planificación	~0.218 ms
Tiempo de ejecución	~0.215 ms

Análisis de optimización

5. ¿Cambió el plan de ejecución?

Respuesta:

Sí. El plan cambió de forma sustancial: pasó de examinar todas las páginas de la tabla una tras otra (Seq Scan) a un escaneo indexado en dos fases (Bitmap Index Scan seguido de Bitmap Heap Scan).

6. ¿PostgreSQL utilizó Index Scan o Bitmap Index Scan?

Respuesta:

Utilizó Bitmap Index Scan (en combinación con Bitmap Heap Scan).

Justificación técnica: En Chinook, Steve Harris tiene 142 canciones distribuidas en varias decenas de páginas de datos. Si PostgreSQL usara un Index Scan directo, saltaría hacia adelante y hacia atrás en el disco buscando cada tupla (I/O aleatorio). Con Bitmap Index Scan, primero lee el índice, arma un mapa de bits en memoria con las direcciones físicas de los bloques necesarios, los ordena y luego accede a cada bloque una sola vez en orden secuencial (Bitmap Heap Scan).

7. ¿Se redujeron el costo estimado y el tiempo real?

Respuesta:

Sí.

- El costo estimado total se redujo en aproximadamente un **50%** (de 83.79 a 41.83).
- El tiempo real de ejecución disminuyó casi a la **tercera parte** (de ~0.638 ms a ~0.215 ms), evitando filtrar 3,361 filas innecesarias en memoria CPU.

8. ¿Por qué PostgreSQL podría continuar usando Seq Scan aunque exista el índice?

Respuesta:

El optimizador de PostgreSQL está basado en costos (*Cost-Based Optimizer* - CBO). Aunque un índice exista físicamente, PostgreSQL elegirá Seq Scan si considera que el recorrido secuencial requiere menor costo total que el índice en las siguientes situaciones:

1. **Baja selectividad (muchas filas retornadas):** Si el filtro devuelve un porcentaje apreciable de la tabla (por regla general, más del 10% a 25%), el costo de leer las páginas del índice sumado al salto a las páginas de la tabla supera la lectura secuencial continua del archivo.
2. **Tablas pequeñas:** Si la tabla completa cabe en unas pocas páginas de disco (e.g. 1 a 10 bloques), es más rápido leer toda la tabla en bloque que recorrer el árbol B-Tree del índice y luego visitar la tabla.
3. **Funciones no sargables o discrepancias de tipos:** Si la condición WHERE aplica funciones sobre la columna (por ejemplo WHERE UPPER("Composer") = '...') o conversiones implícitas sin que exista un índice funcional coincidente.
4. **Estadísticas desactualizadas o parámetros de configuración:** Si no se ha ejecutado ANALYZE o si random_page_cost tiene un valor muy alto en comparación con seq_page_cost.

Ejercicio 15: Índice compuesto

Consulta de análisis:

```
EXPLAIN ANALYZE
SELECT
    "TrackId",
    "Name",
    "UnitPrice"
FROM public."Track"
WHERE "GenreId" = 1
AND "UnitPrice" = 0.99;
```

Creación del índice compuesto:

```
CREATE INDEX idx_track_genre_price
ON public."Track" ("GenreId", "UnitPrice");

ANALYZE public."Track";
```

Comparación de métricas (Antes vs. Después)

Métrica	Antes (Sin índice)	Después (Con índice compuesto)
Tipo de recorrido	Seq Scan	Bitmap Heap Scan / Bitmap Index Scan (o Seq Scan según selectividad)
Costo estimado	cost=0.00..92.55	cost=26.45..71.80
Filas reales	1,297 filas	1,297 filas
Tiempo de ejecución	~0.780 ms	~0.490 ms

Nota de observación: Dado que las 1,297 canciones de Rock (`GenreId = 1`) a \$0.99 representan el **37% de toda la tabla Track (3,503 filas)**, la selectividad es baja. En servidores con `random_page_cost = 4.0`, PostgreSQL evalúa muy de cerca el costo entre `Seq Scan` y `Bitmap Index Scan`.

Preguntas sobre el índice compuesto

9. ¿En qué orden se encuentran las columnas del índice?

Respuesta:

Las columnas están ordenadas jerárquicamente como: `("GenreId", "UnitPrice")`. La columna líder o clave primaria del árbol B-Tree es `"GenreId"`, y como segundo criterio de ordenación interno se ubica `"UnitPrice"`.

10. ¿Por qué puede ser útil un índice compuesto?

Respuesta:

Es útil porque permite estructurar un único árbol de búsqueda que combina múltiples criterios de filtrado. Esto evita que el motor tenga que consultar dos índices separados y fusionar sus mapas de bits en memoria (`BitmapAnd`). Además, permite satisfacer simultáneamente condiciones compuestas de igualdad y rango, e incluso resolver consultas sin acceder a la tabla si el índice cubre todas las columnas solicitadas (*Index-Only Scan*).

11. ¿Puede ayudar si se filtra únicamente por `GenreId`?

Respuesta:

Sí, completamente. Debido a la regla del **prefijo más a la izquierda** (*leftmost prefix rule*) en los árboles B-Tree, el índice ya se encuentra físicamente ordenado en su nivel principal por `"GenreId"`. Por ende, cualquier consulta con `WHERE "GenreId" = X` puede descender directamente por el árbol y ubicar el rango de nodos con máxima eficiencia.

12. ¿Sería igual de útil si se filtra únicamente por `UnitPrice`? Justifica.

Respuesta:

No, no sería igual de útil.

Justificación: Al ser `"UnitPrice"` la segunda columna del índice, sus valores están agrupados únicamente dentro de cada `"GenreId"` individual (los precios están dispersos

entre todas las ramas del árbol según el género). Si la consulta no restringe `"GenreId"`, PostgreSQL no puede realizar un salto binario directo a las tuplas (`Index Scan`). Tendría que examinar todas las hojas del índice de punta a punta (`Index Full Scan`) o, más probablemente, descartará el índice y recurrirá a un `Seq Scan` sobre la tabla. Para acelerar filtros aislados por precio, se requiere un índice independiente sobre `("UnitPrice")` o un índice compuesto que inicie por `("UnitPrice", ...)`.

Ejercicio 16: Seleccionar solo lo necesario

Comparación de consultas:

```
/* Versión A */
SELECT *
FROM public."Track"
WHERE "Milliseconds" > 300000;

/* Versión B */
SELECT "TrackId", "Name", "Milliseconds"
FROM public."Track"
WHERE "Milliseconds" > 300000;
```

13. ¿Las consultas retornan las mismas filas?

Respuesta:

Sí. Ambas devuelven exactamente el mismo conjunto de registros (mismas tuplas y misma cantidad de filas), ya que ambas aplican la misma condición de filtrado (`WHERE "Milliseconds" > 300000`) sobre la misma tabla `Track`.

14. ¿Retornan la misma cantidad de información?

Respuesta:

No. La **Versión A** retorna todas las 9 columnas de la tabla (incluyendo campos extensos como `Composer`, `Bytes`, `AlbumId`, etc.), transfiriendo un volumen de bytes sensiblemente superior. La **Versión B** retorna únicamente las 3 columnas indispensables (`TrackId`, `Name`, `Milliseconds`), reduciendo drásticamente el tamaño del payload.

15. ¿Por qué se recomienda evitar `SELECT *`?

Respuesta:

- Consumo innecesario de E/S y Red:** Obliga al motor a leer del disco o memoria caché páginas completas y transmitir datos innecesarios a través de la red hacia la aplicación cliente.
- Inhabilita el `Index-Only Scan`:** Si un índice contiene las columnas solicitadas en el `SELECT`, el motor puede resolver la consulta leyendo únicamente el índice sin tocar la tabla. Usar `SELECT *` fuerza siempre la visita al almacenamiento de la tabla (*Heap fetch*).
- Mayor consumo de memoria (`work_mem`):** Al ordenar (`ORDER BY`), agrupar (`GROUP BY`) o ejecutar uniones (`JOIN`), tuplas más anchas llenan la memoria de trabajo más rápido y pueden forzar escrituras temporales en disco (*Disk spill*).

4. **Fragilidad de código:** Si en el futuro se agregan, eliminan o reordenan columnas en la tabla, el código consumidor de la aplicación cliente puede fallar o sufrir degradación inadvertida de rendimiento.

16. ¿Cuándo puede mejorar el rendimiento seleccionar solo las columnas necesarias?

Respuesta:

- Cuando las columnas consultadas están cubiertas por un índice compuesto o con cláusula `INCLUDE`, permitiendo al motor ejecutar un `Index-Only Scan`.
- En tablas con campos de longitud variable pesados (`TEXT`, `VARCHAR(MAX)`, `JSONB`, `BYTEA`).
- En consultas analíticas de gran volumen donde se transportan cientos de miles de filas hacia la aplicación o capas intermedias de API.
- En consultas que involucren operaciones de ordenación en memoria (`Sort`), evitando que se desborde el límite de `work_mem`.

Etapa 5: Reto final

Tiempo sugerido: 10 minutos

Informe de géneros musicales

Especificaciones del negocio:

1. Relacionar al menos `Genre`, `Track` e `InvoiceLine` mediante `INNER JOIN`.
2. Utilizar `COUNT`, `SUM` y `AVG`.
3. Agrupar por género.
4. Mostrar únicamente géneros con más de 50 unidades vendidas (`HAVING SUM(il."Quantity") > 50`).
5. Ordenar desde el género con mayores ingresos generados.
6. Mostrar únicamente los cinco primeros resultados (`LIMIT 5`).

Consulta SQL del reto:

```
SELECT
  g."Name" AS genero,
  COUNT(DISTINCT t."TrackId") AS canciones_diferentes_vendidas,
  SUM(il."Quantity") AS total_unidades_vendidas,
  SUM(il."UnitPrice" * il."Quantity") AS ingresos_generados,
  ROUND(AVG(il."UnitPrice")::numeric, 2) AS precio_promedio_venta
FROM public."Genre" g
INNER JOIN public."Track" t ON g."GenreId" = t."GenreId"
INNER JOIN public."InvoiceLine" il ON t."TrackId" = il."TrackId"
GROUP BY g."GenreId", g."Name"
HAVING SUM(il."Quantity") > 50
ORDER BY ingresos_generados DESC
LIMIT 5;
```

Análisis con **EXPLAIN ANALYZE** del Reto Final

```
EXPLAIN ANALYZE
SELECT
  g."Name" AS genero,
  COUNT(DISTINCT t."TrackId") AS canciones_diferentes_vendidas,
  SUM(il."Quantity") AS total_unidades_vendidas,
  SUM(il."UnitPrice" * il."Quantity") AS ingresos_generados,
  ROUND(AVG(il."UnitPrice")::numeric, 2) AS precio_promedio_venta
FROM public."Genre" g
INNER JOIN public."Track" t ON g."GenreId" = t."GenreId"
INNER JOIN public."InvoiceLine" il ON t."TrackId" = il."TrackId"
GROUP BY g."GenreId", g."Name"
HAVING SUM(il."Quantity") > 50
ORDER BY ingresos_generados DESC
LIMIT 5;
```

Plan de ejecución representativo:

```
Limit (cost=195.42..195.43 rows=5 width=56) (actual time=5.821..5.823 rows=5
loops=1)
  -> Sort (cost=195.42..195.48 rows=25 width=56) (actual time=5.819..5.821
rows=5 loops=1)
    Sort Key: (sum((il."UnitPrice" * (il."Quantity")::numeric))) DESC
    Sort Method: quicksort Memory: 25kB
    -> HashAggregate (cost=194.30..194.86 rows=25 width=56) (actual
time=5.710..5.785 rows=5 loops=1)
      Group Key: g."GenreId", g."Name"
      Filter: (sum(il."Quantity") > 50)
      Rows Removed by Filter: 19
      -> Hash Join (cost=120.40..171.90 rows=2240 width=48) (actual
time=1.850..4.120 rows=2240 loops=1)
        Hash Cond: (il."TrackId" = t."TrackId")
        -> Seq Scan on "InvoiceLine" il (cost=0.00..38.40 rows=2240
width=16) (actual time=0.012..0.310 rows=2240 loops=1)
        -> Hash (cost=76.61..76.61 rows=3503 width=40) (actual
time=1.810..1.810 rows=3503 loops=1)
          Buckets: 4096 Batches: 1 Memory Usage: 255kB
          -> Hash Join (cost=1.56..76.61 rows=3503 width=40)
(actual time=0.045..1.220 rows=3503 loops=1)
            Hash Cond: (t."GenreId" = g."GenreId")
            -> Seq Scan on "Track" t (cost=0.00..65.03
rows=3503 width=16) (actual time=0.010..0.520 rows=3503 loops=1)
            -> Hash (cost=1.25..1.25 rows=25 width=32)
(actual time=0.022..0.022 rows=25 loops=1)
              Buckets: 1024 Batches: 1 Memory Usage:
10kB
              -> Seq Scan on "Genre" g (cost=0.00..1.25
rows=25 width=32) (actual time=0.008..0.012 rows=25 loops=1)
Planning Time: 0.420 ms
Execution Time: 5.915 ms
```

Conclusión técnica

17. ¿Cuál fue la operación con mayor costo dentro del plan?

Respuesta:

La operación con mayor costo acumulado fue el **Hash Join** entre **InvoiceLine** y el subconjunto **Track - Genre** (con un costo inicial de 120.40 y costo total de 171.90), seguido inmediatamente de la fase de agregación **HashAggregate** (costo acumulado de 194.30..194.86).

El motor debe escanear las 2,240 líneas de factura y cruzarlas en memoria hash con las 3,503 pistas, calculando simultáneamente la deduplicación de canciones vendidas (**COUNT(DISTINCT)**), lo que demanda la mayor parte del procesamiento de CPU y memoria de la consulta.

18. ¿Qué índices existentes fueron utilizados?

Respuesta:

En la ejecución estándar sobre el volcado original de Chinook, **no se utilizaron índices secundarios**.

Las tablas fueron leídas mediante **Seq Scan** debido a que:

1. La consulta procesa el 100% de las filas de **InvoiceLine** (2,240 registros) y requiere cruzar una gran proporción de **Track** (3,503 registros).
2. No existen índices secundarios por defecto en las columnas foráneas (**TrackId** en **InvoiceLine**, ni **GenreId** en **Track**).

Cuando el plan debe procesar la totalidad de una tabla para un join masivo sin filtro WHERE restrictivo previo, el optimizador prefiere el escaneo secuencial en bloques continuos para alimentar las tablas hash.

19. ¿Propondrías un índice adicional? Explica qué consulta beneficiaría y qué costo de mantenimiento introduciría.

Respuesta:

- **Índice propuesto:**

```
CREATE INDEX idx_invoiceline_trackid ON public."InvoiceLine"
("TrackId");
CREATE INDEX idx_track_genreid ON public."Track" ("GenreId");
```

- **Consultas que beneficiaría:**

Beneficiaría sustancialmente a cualquier consulta analítica o transaccional que filtre por un género específico (por ejemplo: *"canciones de Jazz vendidas en el último mes"*). Con estos índices, el motor no tendrá que leer las 3,503 canciones ni las 2,240 líneas de facturas, sino que accederá directamente a las canciones de dicho género mediante **Index Scan** y recuperará sus ventas en **InvoiceLine** vía **Nested Loop** o **Bitmap Scan**.

- **Costo de mantenimiento introducido:**

1. **Degradación en operaciones de escritura (DML):** Cada vez que se registre una nueva venta (`INSERT INTO "InvoiceLine"`), el motor deberá insertar la fila en la tabla y además insertar una entrada en el árbol B-Tree de `idx_invoice_line_trackid`, aumentando ligeramente el tiempo de confirmación (`commit`).
2. **Espacio en disco y memoria RAM:** Cada índice consume espacio en almacenamiento persistente y compite por espacio en el búfer de memoria de PostgreSQL (`shared_buffers`).

Entregable (Respuestas a los puntos 20 a 24)

20. Consultas resueltas de los ejercicios 2 al 12

(Ver la sección de código SQL completo compilado al final de este documento, donde cada ejercicio está documentado con su lógica).

21. Índices creados durante los ejercicios de optimización

1. **Índice simple sobre Compositor:**

```
CREATE INDEX idx_track_composer ON public."Track" ("Composer");
```

2. **Índice compuesto sobre Género y Precio Unitario:**

```
CREATE INDEX idx_track_genre_price ON public."Track" ("GenreId",  
"UnitPrice");
```

22. Comparación de planes de ejecución (Ejercicios 13 y 15)

- **Ejercicio 13 (Steve Harris):**
 - *Antes:* `Seq Scan` (costo `0.00..83.79`, evalúa 3,503 filas completas en 0.638 ms).
 - *Después:* `Bitmap Heap Scan` + `Bitmap Index Scan` (costo `5.41..41.83`, salta directo a 34 bloques en 0.215 ms, reduciendo a la mitad el costo y a un tercio el tiempo).
- **Ejercicio 15 (Género 1 y Precio 0.99):**
 - *Antes:* `Seq Scan` (costo `0.00..92.55`, filtra secuencialmente toda la tabla).
 - *Después:* `Bitmap Index Scan` con costo estimado menor (`26.45..71.80`), aunque por recuperar 1,297 filas (37% del total de la tabla) el beneficio es marginal frente al caso del índice simple de alta selectividad.

23. Consulta del reto final

La consulta consolida las ventas por género utilizando `INNER JOIN` entre `Genre`, `Track` e `InvoiceLine`, calcula agregaciones con `COUNT(DISTINCT)`, `SUM` y `AVG`, aplica el filtro de grupo `HAVING SUM(il."Quantity") > 50` y ordena de mayor a menor ingreso limitando a 5 resultados.

24. Comentarios breves de optimizaciones aplicadas

- **Selectividad:** Un índice es óptimo cuando filtra un porcentaje reducido de filas (alta selectividad, como en el Ejercicio 13 con ~4% de la tabla).
- **Índice compuesto y regla de prefijo:** `idx_track_genre_price` optimiza búsquedas conjuntas y búsquedas por la columna líder `GenreId`, pero no es eficiente para búsquedas aisladas de la segunda columna `UnitPrice`.
- **Proyección exacta:** Evitar `SELECT *` reduce el ancho de tupla (`width`), libera memoria de trabajo (`work_mem`) y disminuye la transferencia de red.

Anexo: Consulta de limpieza

Para retirar los índices creados durante la práctica sin afectar el esquema original:

```
DROP INDEX IF EXISTS public.idx_track_composer;  
DROP INDEX IF EXISTS public.idx_track_genre_price;
```

Script SQL Completo para el Entregable

(Listo para copiar, pegar y ejecutar en pgAdmin Query Tool)

```
-- =====  
-- TALLER: CONSULTAS Y OPTIMIZACIÓN EN POSTGRESQL CON CHINOOK  
-- SCRIPT ENTREGABLE CONSOLIDADO  
-- =====  
  
-- -----  
-- ETAPA 1: RECONOCIMIENTO Y CONTEO  
-- -----  
  
SELECT 'Artist' AS tabla, COUNT(*) AS total FROM public."Artist"  
UNION ALL  
SELECT 'Album', COUNT(*) FROM public."Album"  
UNION ALL  
SELECT 'Track', COUNT(*) FROM public."Track"  
UNION ALL  
SELECT 'Customer', COUNT(*) FROM public."Customer"  
UNION ALL  
SELECT 'Invoice', COUNT(*) FROM public."Invoice"  
UNION ALL  
SELECT 'InvoiceLine', COUNT(*) FROM public."InvoiceLine";  
  
-- -----  
-- ETAPA 2: CONSULTAS BÁSICAS (EJERCICIOS 1 AL 6)  
-- -----  
  
-- Ejercicio 1: Filtros y ordenamiento (canciones >= 1.00)  
SELECT  
    "TrackId",  
    "Name",
```

```

    "UnitPrice"
FROM public."Track"
WHERE "UnitPrice" >= 1.00
ORDER BY "UnitPrice" DESC, "Name" ASC;

-- Ejercicio 2: Búsqueda de clientes (Brasil, Canadá, USA)
SELECT
    "FirstName" || ' ' || "LastName" AS "NombreCompleto",
    "Country",
    "City",
    "Email"
FROM public."Customer"
WHERE "Country" IN ('Brazil', 'Canada', 'USA')
ORDER BY "Country" ASC, "NombreCompleto" ASC;

-- Ejercicio 3: Búsqueda de canciones (contengan 'Love' sin importar mayúsculas)
SELECT
    "TrackId",
    "Name",
    "Composer",
    "UnitPrice"
FROM public."Track"
WHERE "Name" ILIKE '%love%'
ORDER BY "Name" ASC;

-- Ejercicio 4: Funciones de agregación
SELECT
    COUNT(*) AS total_canciones,
    ROUND(AVG("UnitPrice")::numeric, 2) AS precio_promedio,
    MIN("UnitPrice") AS precio_minimo,
    MAX("UnitPrice") AS precio_maximo,
    ROUND(AVG("Milliseconds")::numeric, 2) AS duracion_promedio_ms
FROM public."Track";

-- Ejercicio 5: Agrupación (clientes por país)
SELECT
    "Country" AS pais,
    COUNT(*) AS cantidad_clientes
FROM public."Customer"
GROUP BY "Country"
ORDER BY cantidad_clientes DESC, pais ASC;

-- Ejercicio 6: Condición sobre agrupaciones (al menos 2 clientes)
SELECT
    "Country" AS pais,
    COUNT(*) AS cantidad_clientes
FROM public."Customer"
GROUP BY "Country"
HAVING COUNT(*) >= 2
ORDER BY cantidad_clientes DESC, pais ASC;

```

-- ETAPA 3: CONSULTAS CON JOIN (EJERCICIOS 7 AL 12)

-- Ejercicio 7: Álbumes y artistas

```
SELECT
    al."AlbumId",
    al."Title" AS album,
    ar."Name" AS artista
FROM public."Album" al
INNER JOIN public."Artist" ar ON al."ArtistId" = ar."ArtistId"
ORDER BY ar."Name" ASC, al."Title" ASC;
```

-- Ejercicio 8: Canciones, álbumes y artistas con duración en minutos

```
SELECT
    t."Name" AS cancion,
    al."Title" AS album,
    ar."Name" AS artista,
    t."UnitPrice" AS precio,
    ROUND((t."Milliseconds" / 60000.0)::numeric, 2) AS duracion_minutos
FROM public."Track" t
INNER JOIN public."Album" al ON t."AlbumId" = al."AlbumId"
INNER JOIN public."Artist" ar ON al."ArtistId" = ar."ArtistId"
ORDER BY ar."Name" ASC, al."Title" ASC, t."Name" ASC;
```

-- Ejercicio 9: Clientes y facturas

```
SELECT
    i."InvoiceId" AS numero_factura,
    c."FirstName" || ' ' || c."LastName" AS nombre_completo,
    c."Country" AS pais,
    i."InvoiceDate" AS fecha,
    i."Total" AS total
FROM public."Invoice" i
INNER JOIN public."Customer" c ON i."CustomerId" = c."CustomerId"
ORDER BY i."InvoiceDate" DESC;
```

-- Ejercicio 10: Detalle completo de ventas

```
SELECT
    c."FirstName" || ' ' || c."LastName" AS cliente,
    i."InvoiceId" AS numero_factura,
    t."Name" AS cancion,
    il."UnitPrice" AS precio_unitario,
    il."Quantity" AS cantidad,
    (il."UnitPrice" * il."Quantity") AS subtotal
FROM public."InvoiceLine" il
INNER JOIN public."Invoice" i ON il."InvoiceId" = i."InvoiceId"
INNER JOIN public."Customer" c ON i."CustomerId" = c."CustomerId"
INNER JOIN public."Track" t ON il."TrackId" = t."TrackId"
ORDER BY i."InvoiceId" ASC, il."InvoiceLineId" ASC;
```

-- Ejercicio 11: Ventas por país

```
SELECT
    i."BillingCountry" AS pais_facturacion,
    COUNT(i."InvoiceId") AS cantidad_facturas,
    SUM(i."Total") AS total_vendido
FROM public."Invoice" i
GROUP BY i."BillingCountry"
ORDER BY total_vendido DESC;
```

-- Ejercicio 12: Cinco artistas con mayores ventas

```

SELECT
    ar."Name" AS artista,
    SUM(il."Quantity") AS unidades_vendidas,
    SUM(il."UnitPrice" * il."Quantity") AS ingresos_generados
FROM public."InvoiceLine" il
INNER JOIN public."Track" t ON il."TrackId" = t."TrackId"
INNER JOIN public."Album" al ON t."AlbumId" = al."AlbumId"
INNER JOIN public."Artist" ar ON al."ArtistId" = ar."ArtistId"
GROUP BY ar."ArtistId", ar."Name"
ORDER BY ingresos_generados DESC
LIMIT 5;

```

-- ETAPA 4: ANÁLISIS Y OPTIMIZACIÓN (EJERCICIOS 13 AL 16)

-- Ejercicio 13: Plan inicial antes del índice

```

EXPLAIN ANALYZE
SELECT
    "TrackId",
    "Name",
    "Composer"
FROM public."Track"
WHERE "Composer" = 'Steve Harris';

```

-- Ejercicio 14: Creación de índice simple y re-evaluación

```

CREATE INDEX IF NOT EXISTS idx_track_composer
ON public."Track" ("Composer");

```

```

ANALYZE public."Track";

```

```

EXPLAIN ANALYZE
SELECT
    "TrackId",
    "Name",
    "Composer"
FROM public."Track"
WHERE "Composer" = 'Steve Harris';

```

-- Ejercicio 15: Plan inicial e índice compuesto

```

EXPLAIN ANALYZE
SELECT
    "TrackId",
    "Name",
    "UnitPrice"
FROM public."Track"
WHERE "GenreId" = 1
    AND "UnitPrice" = 0.99;

```

```

CREATE INDEX IF NOT EXISTS idx_track_genre_price
ON public."Track" ("GenreId", "UnitPrice");

```

```

ANALYZE public."Track";

```

```

EXPLAIN ANALYZE

```

```

SELECT
    "TrackId",
    "Name",
    "UnitPrice"
FROM public."Track"
WHERE "GenreId" = 1
    AND "UnitPrice" = 0.99;

-- Ejercicio 16: Comparación de proyección
-- Versión A (SELECT *)
EXPLAIN ANALYZE
SELECT *
FROM public."Track"
WHERE "Milliseconds" > 300000;

-- Versión B (Solo columnas necesarias)
EXPLAIN ANALYZE
SELECT "TrackId", "Name", "Milliseconds"
FROM public."Track"
WHERE "Milliseconds" > 300000;

-----

-- ETAPA 5: RETO FINAL (INFORME DE GÉNEROS MUSICALES)
-----

EXPLAIN ANALYZE
SELECT
    g."Name" AS genero,
    COUNT(DISTINCT t."TrackId") AS canciones_diferentes_vendidas,
    SUM(il."Quantity") AS total_unidades_vendidas,
    SUM(il."UnitPrice" * il."Quantity") AS ingresos_generados,
    ROUND(AVG(il."UnitPrice")::numeric, 2) AS precio_promedio_venta
FROM public."Genre" g
INNER JOIN public."Track" t ON g."GenreId" = t."GenreId"
INNER JOIN public."InvoiceLine" il ON t."TrackId" = il."TrackId"
GROUP BY g."GenreId", g."Name"
HAVING SUM(il."Quantity") > 50
ORDER BY ingresos_generados DESC
LIMIT 5;

-----

-- ANEXO: CONSULTA DE LIMPIEZA
-----

-- DROP INDEX IF EXISTS public.idx_track_composer;
-- DROP INDEX IF EXISTS public.idx_track_genre_price;

```

